

Open Source Project

Enchanter

Software Requirements Specification (SRS)

Harness Differentiation & Feature Roadmap

Version 0.1.0-draft

MIT License

Rust

Repository: github.com/andrewthecodertx/enchanter

Document Status: **Draft — Open for Community Review**

Date: 2026-05-24

This document is released under the same MIT license as the Enchanter project.
Contributions, corrections, and discussion are welcome via GitHub Issues and Pull Requests.

Revision History

Version	Date	Author(s)	Description
0.1.0-draft	2026-05-24	Andrew S. Erwin + community	Initial SRS draft for community review

Keyword Convention

Keyword	Meaning
SHALL	Mandatory requirement. Must be implemented for the feature to be considered complete.
SHOULD	Strongly recommended. May be deferred with documented rationale.
MAY	Optional. Included for completeness; community may choose to implement.

Note for contributors: Each requirement carries a unique ID in the form `REQ-<AREA>-<NNN>` .
When opening a PR or Issue, reference the relevant ID(s) to keep discussion traceable.

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions & Abbreviations
 - 1.4 References
 - 1.5 Document Overview
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions (Summary)
 - 2.3 User Classes & Characteristics
 - 2.4 Operating Environment
 - 2.5 Design & Implementation Constraints
 - 2.6 Assumptions & Dependencies
3. System Features & Requirements
 - 3.1 Inspectability-First Harness
 - 3.2 Reproducible Runs
 - 3.3 Project Overlays
 - 3.4 MCP Security Posture
 - 3.5 Skill Packages
 - 3.6 Harness Evals
 - 3.7 Terminal UX / TUI

3.8 Memory Governance

4. External Interface Requirements
5. Non-Functional Requirements
6. Suggested Architecture
7. Suggested Libraries (Rust Crates)
8. File & Format Specifications
9. Milestones & Traceability Matrix
10. Open Questions
11. Contributing

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, architectural, and interface requirements for the **Enchanter** open-source project. It is intended to guide contributors, maintainers, and community reviewers in implementing a set of differentiating features that establish Enchanter as the most inspectable, reproducible, and pragmatic terminal-native AI agent harness available.

This document follows a structure inspired by IEEE 830 / ISO/IEC 29148, adapted for an open-source project where community participation and transparency are first-class values.

1.2 Scope

Enchanter is a focused AI agent harness: a single Rust binary that connects any OpenAI-compatible model provider to a local SOUL persona, persistent memory, drop-in skills, and MCP-based tools — with zero runtime dependencies.

This SRS covers eight feature areas that differentiate Enchanter from competing agent frameworks and harnesses:

1. Inspectability-first harness (prompt diff, budget view)
2. Reproducible runs (record + replay)
3. Project overlays (per-repo harness scoping)
4. MCP security posture (capability policies + consent UX)
5. Skill packages (ecosystem without framework overhead)
6. Harness evals (measure harness quality, not just model quality)
7. Terminal UX / optional TUI
8. Memory governance (channels, pin/redact, optional encryption)

This SRS does **not** cover: model training, provider infrastructure, IDE integrations, or mobile/GUI clients.

1.3 Definitions & Abbreviations

Term	Definition
Agent harness	The surrounding infrastructure (memory, tools, prompt assembly, environment) that turns a raw LLM into a reliable autonomous agent. "Agent = Model + Harness."
SOUL	The persona/system-prompt file (<code>SOUL.md</code>) that defines the agent's identity and behavior.
Skill / SKILL.md	A markdown file describing a capability the agent can invoke. Compatible with agentskills.io format.
Memory	Persistent context files in <code>memories/</code> , using the Hermes Agent <code>§</code> -delimited format.
MCP	Model Context Protocol — a standard for connecting AI agents to external tools via stdio or HTTP transports.

REPL	Read-Eval-Print Loop — Enchanter's interactive terminal session.
TUI	Terminal User Interface — a richer, pane-based terminal experience (optional).
Overlay	A project-local <code>.enchanter/</code> directory that supplements or overrides the global <code>~/.enchanter/</code> config.
Harness eval	A task-based evaluation of harness configuration quality (memory strategy, toolsets, prompt assembly), distinct from model benchmarks.
JSONL	JSON Lines — a newline-delimited JSON format used for run recordings.
REQ-ID	Unique requirement identifier in the form <code>REQ-<AREA>-<NNN></code> .
SRS	Software Requirements Specification (this document).
OSS	Open-Source Software.

1.4 References

- Enchanter README — github.com/andrewthecodertx/enchanter
- Martin Fowler — "Harness Engineering for Coding Agent Users" — martinfowler.com/articles/harness-engineering.html
- LangChain — "The Anatomy of an Agent Harness" — langchain.com/blog/the-anatomy-of-an-agent-harness
- OpenAI — "Harness Engineering" — openai.com/index/harness-engineering/
- Anthropic — "Harness Design for Long-Running Apps" — anthropic.com/engineering/harness-design-long-running-apps
- agentskills.io — SKILL.md format specification
- Model Context Protocol specification — modelcontextprotocol.io
- IEEE 830-1998 / ISO/IEC 29148:2018 — SRS guidelines

1.5 Document Overview

Section 2 provides overall product context. Section 3 contains the detailed functional requirements organized by feature area. Sections 4–5 cover interface and non-functional requirements. Sections 6–7 provide architectural and library guidance. Sections 8–11 cover formats, milestones, open questions, and contribution guidance.

2. Overall Description

2.1 Product Perspective

Enchanter is a standalone terminal tool, not a library or SDK. It occupies a distinct niche: between "raw curl to an LLM API" and "full agent framework requiring an application scaffold." Its value proposition is:

- **Transparency:** the user always knows what the model sees.
- **Portability:** single binary, works over SSH, no Node/Python/Docker required.
- **Composability:** SOUL, memory, skills, and MCP tools are plain files — easy to version-control, share, and script.
- **Reproducibility:** sessions can be recorded and replayed for debugging and iteration.

Enchanter is compatible with the Hermes Agent memory format and agentskills.io SKILL.md format, enabling users to migrate from or interoperate with those ecosystems.

2.2 Product Functions (Summary)

- Interactive REPL and single-shot (`-p`) modes for LLM interaction
- Layered system prompt assembly (SOUL / CONTEXT / VOLATILE)
- Persistent memory with session summarization
- Drop-in skill discovery and indexing
- MCP tool integration (stdio + HTTP)
- Named provider switching mid-session
- Prompt inspection, diff, and budget analysis
- Session recording and replay
- Project-scoped overlay configuration
- MCP capability policies and consent UX
- Skill package management
- Harness evaluation suite
- Optional TUI mode
- Memory channels, governance commands, and optional encryption

2.3 User Classes & Characteristics

Class	Description	Primary Needs
Solo Developer	Uses Enchanter daily as a terminal AI assistant. Comfortable with CLI tools.	Reliability, fast setup, persistent memory, provider flexibility.
Power User / Tinkerer	Wants to tune harness behavior, inspect prompts, compare configurations.	Prompt diff/budget, harness evals, project overlays, memory governance.
Team / OSS Maintainer	Shares skills, uses project overlays per repo, cares about security and reproducibility.	Skill packages, reproducible runs, MCP security posture, audit logs.

Contributor	Wants to add features, fix bugs, or write skills/tools for the ecosystem.	Clear architecture, documented formats, testable modules, good CONTRIBUTING guide.
-------------	---	--

2.4 Operating Environment

- **Primary:** Linux (x86_64, aarch64), macOS (Apple Silicon + Intel)
- **Secondary:** Windows (WSL2 supported; native Windows is a stretch goal)
- **Terminal:** Any ANSI-compatible terminal; TUI requires 256-color support
- **Network:** Outbound HTTPS to model provider APIs; MCP HTTP servers; local MCP stdio processes
- **Filesystem:** Read/write to `~/.enchanter/` and optionally `.enchanter/` in project directories

2.5 Design & Implementation Constraints

- **Language:** Rust (stable toolchain). No unsafe code without documented justification and review.
- **Distribution:** Single statically-linked binary. No runtime dependencies (no Node, Python, Docker).
- **License:** MIT. All dependencies must be MIT, Apache-2.0, or BSD-compatible.
- **Backwards compatibility:** Existing `~/.enchanter/` layouts (SOUL.md, memories/, skills/, config.yaml) must continue to work without migration steps.
- **Secrets:** API keys and tokens must never appear in logs, recordings (unless explicitly opted in), or terminal output by default.
- **Binary size:** TUI and eval features should be gated behind Cargo feature flags to keep the minimal binary lean.

2.6 Assumptions & Dependencies

- The model provider exposes an OpenAI-compatible chat completions API.
- MCP servers are either locally installable (stdio) or reachable via HTTPS (HTTP transport).
- Users have a working Rust toolchain for building from source.
- The `§`-delimited memory format and SKILL.md format remain stable upstream.

3. System Features & Requirements

3.1 Feature: Inspectability-First Harness

Description

Enchanter assembles the system prompt in three layers (SOUL / CONTEXT / VOLATILE). This feature makes those layers fully auditable: users can see exactly what the model received, how it changed between turns, and how much of the context window each layer consumes.

3.1.1 Prompt Diff

REQ-ID	Keyword	Requirement
REQ- INS-001	SHALL	The system shall provide a <code>/prompt diff</code> REPL command that displays a human-readable diff of the assembled system prompt between the previous turn and the current turn.
REQ- INS-002	SHALL	The system shall provide an <code>enchanter prompt --diff</code> CLI subcommand equivalent to <code>/prompt diff</code> .
REQ- INS-003	SHALL	The diff output shall clearly indicate changes in: memory blocks added/removed, skill index changes, tool list changes (including MCP servers), provider/model changes, and SOUL changes.
REQ- INS-004	SHALL	The diff output shall never display API keys, tokens, or auth headers in plaintext.
REQ- INS-005	SHOULD	The diff should use color-coded output (green for additions, red for removals) when the terminal supports ANSI color.

3.1.2 Prompt Budget View

REQ-ID	Keyword	Requirement
REQ- INS-006	SHALL	The system shall provide a <code>/prompt budget</code> REPL command and <code>enchanter prompt --budget</code> CLI subcommand.
REQ- INS-007	SHALL	The budget view shall display approximate character and estimated token counts per prompt layer: SOUL, CONTEXT, and VOLATILE.
REQ- INS-008	SHOULD	The budget view should display a heuristic suggestion when any layer exceeds a configurable threshold (default: 4,000 tokens estimated).
REQ- INS-009	MAY	The budget view may display a visual bar chart in the terminal representing relative layer sizes.
REQ- INS-010	SHALL	Token estimation shall be clearly labeled as approximate (e.g., "~1,240 tokens estimated") to avoid implying provider-specific accuracy.

3.2 Feature: Reproducible Runs

Description

Users can record a full session to a JSONL file and replay it later — either to reproduce behavior exactly (with stubbed tool outputs) or to re-run with a different model while keeping the same harness inputs. This enables debugging, regression testing, and harness iteration.

3.2.1 Recording

REQ-ID	Keyword	Requirement
REQ-REC-001	SHALL	The system shall support a <code>--record <file.jsonl></code> CLI flag that records the full session to a JSONL file.
REQ-REC-002	SHALL	Each recorded event shall include: event type, monotonic sequence number, UTC timestamp, and event payload.
REQ-REC-003	SHALL	The recording shall capture: config resolution snapshot (redacted by default), prompt layer hashes, user messages, assistant responses, tool calls and results (including MCP), model/provider identifiers, memory writes, and session summary output.
REQ-REC-004	SHALL	The system shall support a <code>--record-redact</code> flag that additionally redacts user-configurable fields from the recording.
REQ-REC-005	SHALL	API keys, tokens, and auth headers shall be redacted from recordings by default, regardless of flags.
REQ-REC-006	SHOULD	The recording format should include a schema version field to support future format evolution.

3.2.2 Replay

REQ-ID	Keyword	Requirement
REQ-REP-001	SHALL	The system shall provide an <code>enchanter replay <file.jsonl></code> subcommand.
REQ-REP-002	SHALL	Replay shall support an <code>--exact</code> mode that errors if the current provider/model does not match the recorded provider/model.
REQ-REP-003	SHALL	Replay shall support a <code>--swap-model <model></code> flag that re-runs with a different model while preserving all harness inputs.
REQ-REP-004	SHALL	Replay shall support a <code>--tools live</code> mode (default) that calls tools live during replay.
REQ-REP-005	SHALL	Replay shall support a <code>--tools stubbed</code> mode that uses recorded tool outputs for deterministic replay without live tool calls.
REQ-REP-006	SHOULD	Replay output should be diffable against the original recording to highlight behavioral changes.

3.3 Feature: Project Overlays

Description

A project-local `.enchanter/` directory supplements or overrides the global `~/.enchanter/` configuration, enabling per-repository agent behavior without breaking the global setup.

REQ-ID	Keyword	Requirement
REQ-OVR-001	SHALL	The system shall discover a <code>.enchanter/</code> directory by searching the current directory and its parents up to the filesystem root or a VCS root (whichever is closer).
REQ-OVR-002	SHALL	The system shall apply the following precedence order (highest to lowest): environment variables, project overlay config, global config, compiled defaults.
REQ-OVR-003	SHALL	The project overlay shall support: <code>SOUL.md</code> addendum or override, <code>config.yaml</code> overlay, <code>memories/</code> , <code>skills/</code> , and MCP server definitions.
REQ-OVR-004	SHALL	The system shall provide a <code>/scope global project both</code> REPL command to switch the active scope at runtime.
REQ-OVR-005	SHALL	When <code>/scope both</code> is active, the merge strategy for conflicting memory IDs and skill names shall be deterministic and documented (project takes precedence).
REQ-OVR-006	SHOULD	The <code>/config</code> command should display which values came from which source (env, project, global, default).
REQ-OVR-007	MAY	The system may support an <code>enchanter init --project</code> command to scaffold a <code>.enchanter/</code> directory in the current directory.

3.4 Feature: MCP Security Posture

Description

MCP tools can perform powerful operations (filesystem access, network calls, process spawning). Enchanter shall enforce capability policies at the harness layer and provide a clear consent UX so users are never surprised by tool behavior.

3.4.1 Capability Policy

REQ-ID	Keyword	Requirement
REQ-SEC-001	SHALL	The system shall support a capability policy configuration in <code>config.yaml</code> that defines per-tool or per-server permissions.
REQ-SEC-002	SHALL	Filesystem capability policies shall support: allowed paths (allowlist), read-only flag, and write flag.
REQ-SEC-003	SHALL	Network capability policies shall support: allowed domain allowlist and a global network-off flag.

REQ- SEC-004	SHALL	Process/exec capability policies shall support explicit allow/deny per MCP server.
REQ- SEC-005	SHALL	Policy enforcement shall occur at the harness layer before the tool call is dispatched, regardless of model instructions.

3.4.2 Consent UX

REQ-ID	Keyword	Requirement
REQ- SEC-006	SHALL	The first use of a tool with elevated capability (write, network, exec) shall trigger an interactive consent prompt in the REPL.
REQ- SEC-007	SHALL	The consent prompt shall offer: Allow once / Allow for session / Allow for project / Deny.
REQ- SEC-008	SHALL	Consent decisions of "Allow for project" shall be stored in the project overlay consent file; "Allow for session" decisions shall not be persisted.
REQ- SEC-009	SHALL	In non-interactive mode (<code>-p</code> flag), the system shall default to Deny for any tool requiring consent that has not been pre-approved in config or consent store.
REQ- SEC-010	SHOULD	All consent decisions should be written to an append-only audit log at <code>~/.enchanter/audit.log</code> .
REQ- SEC-011	MAY	The audit log may be included in run recordings when <code>--record</code> is active.

3.5 Feature: Skill Packages

Description

Skills are currently drop-in SKILL.md files. This feature adds metadata normalization and a lightweight package management CLI so skills can be shared, versioned, and validated without introducing a heavy registry dependency.

3.5.1 Skill Metadata

REQ-ID	Keyword	Requirement
REQ- SKL-001	SHALL	The system shall define and document a minimal metadata front-matter section for SKILL.md files, including: name, version, required_tools (list), description, and examples.
REQ- SKL-002	SHALL	Metadata shall be backwards-compatible: SKILL.md files without metadata front-matter shall continue to work as before.
REQ- SKL-003	SHOULD	Metadata should use YAML front-matter delimited by <code>---</code> at the top of the SKILL.md file.

3.5.2 Skill Management Commands

REQ-ID	Keyword	Requirement
REQ-SKL-004	SHALL	The system shall provide <code>enchanter skill list</code> to display all discovered skills with name, version, and required tools.
REQ-SKL-005	SHALL	The system shall provide <code>enchanter skill add <source></code> where source is a local path or a git URL.
REQ-SKL-006	SHALL	The system shall provide <code>enchanter skill remove <name></code> .
REQ-SKL-007	SHALL	The system shall provide <code>enchanter skill doctor</code> that validates all installed skills for: valid metadata, required tools availability, and format correctness.
REQ-SKL-008	SHOULD	When adding a skill from a git URL, the system should record the commit hash for pinning and display it to the user.
REQ-SKL-009	MAY	The system may support a future registry identifier format (e.g., <code>enchanter skill add registry:skill-name</code>) as a placeholder for a community registry.

3.6 Feature: Harness Evals

Description

Harness evals measure the quality of the harness configuration (memory strategy, toolsets, prompt assembly) independently of model quality. This is distinct from model benchmarks and enables users to quantify improvements from harness changes.

3.6.1 Built-in Eval Suite

REQ-ID	Keyword	Requirement
REQ-EVL-001	SHALL	The system shall ship a built-in eval suite accessible via <code>enchanter eval run</code> .
REQ-EVL-002	SHALL	The eval suite shall include tasks covering: memory recall (short-term and long-term), skill selection accuracy, tool-use correctness (using a mock tool), multi-step planning consistency, and prompt assembly correctness.
REQ-EVL-003	SHALL	Each eval task shall have a documented expected behavior and a pass/fail or scored rubric.
REQ-EVL-004	SHALL	The eval runner shall output results as both a human-readable table and a machine-readable JSON summary.

3.6.2 Scorecards & Comparison

REQ-ID	Keyword	Requirement
	SHOULD	

REQ- EVL-005		The system should support <code>enchanter eval compare <run-a.json> <run-b.json></code> to diff two scorecard outputs.
REQ- EVL-006	SHOULD	Eval runs should be saveable to a named file for later comparison.
REQ- EVL-007	MAY	The system may support community-contributed eval tasks via the same SKILL.md-style drop-in mechanism.
REQ- EVL-008	SHALL	Eval tasks shall use mock/stubbed tool outputs by default to ensure reproducibility independent of live tool availability.

3.7 Feature: Terminal UX / Optional TUI

Description

The REPL remains the primary interface. An optional TUI mode provides a richer pane-based experience for users who want simultaneous visibility into chat, tool traces, memory, and prompt layers.

3.7.1 REPL Enhancements

REQ-ID	Keyword	Requirement
REQ- UX-001	SHALL	All new REPL commands introduced by this SRS shall appear in <code>/help</code> output.
REQ- UX-002	SHALL	Existing REPL commands shall not be broken or removed by any new feature implementation.
REQ- UX-003	SHOULD	The REPL should display a brief status line (model, provider, scope, memory count) on startup.

3.7.2 Optional TUI Mode

REQ-ID	Keyword	Requirement
REQ- UX-004	SHALL	The TUI mode shall be gated behind a Cargo feature flag (<code>--features tui</code>) and not included in the default binary.
REQ- UX-005	SHALL	The TUI shall be activated via <code>enchanter --tui</code> .
REQ- UX-006	SHALL	The TUI shall include panes for: Chat, Tool Trace (live), Memory View, Prompt Layers/Diff, and Provider Status.
REQ- UX-007	SHALL	The TUI shall support keyboard shortcuts for: retry last message, undo last exchange, switch provider, toggle streaming.
REQ- UX-008	SHALL	The TUI shall function correctly over SSH connections and in standard 256-color terminals.

SHOULD

REQ- UX-009		The TUI should degrade gracefully to REPL mode if the terminal does not support required capabilities.
----------------	--	---

3.8 Feature: Memory Governance

Description

Persistent memory is a core Enchanter feature. This feature adds user-controlled channels, governance commands, and optional encryption so memory remains predictable and safe rather than an opaque "magic soup."

3.8.1 Memory Channels

REQ-ID	Keyword	Requirement
REQ- MEM-001	SHALL	The system shall support named memory channels. Default channels: <code>facts</code> , <code>preferences</code> , <code>projects</code> , <code>people</code> , <code>security</code> .
REQ- MEM-002	SHALL	Users shall be able to define custom channels in <code>config.yaml</code> .
REQ- MEM-003	SHALL	Memory without an explicit channel assignment shall default to a <code>general</code> channel for backwards compatibility.

3.8.2 Governance Commands

REQ-ID	Keyword	Requirement
REQ- MEM-004	SHALL	The system shall provide <code>/memory list [channel]</code> to display memory entries, optionally filtered by channel.
REQ- MEM-005	SHALL	The system shall provide <code>/memory promote <id> <channel></code> to move a memory entry to a different channel.
REQ- MEM-006	SHALL	The system shall provide <code>/memory redact <id></code> to permanently remove a memory entry.
REQ- MEM-007	SHALL	The system shall provide <code>/memory pin <id></code> to mark a memory entry as pinned, preventing it from being summarized away during session summarization.
REQ- MEM-008	SHOULD	The system should provide <code>enchanter memory</code> CLI subcommands mirroring the REPL commands above for scripting use.

3.8.3 Encryption at Rest (Optional)

REQ-ID	Keyword	Requirement
REQ- MEM-009	MAY	The system may support optional encryption of the <code>memories/</code> directory, gated behind a Cargo feature flag (<code>--features memory-encryption</code>).
REQ- MEM-010	SHALL	If encryption is implemented, it shall be opt-in and clearly documented, with no silent data loss on key mismatch.

REQ- MEM-011	SHOULD	If encryption is implemented, it should support key supply via environment variable or OS keyring integration.
-----------------	---------------	---

REQ- MEM-012	SHOULD	The <code>age</code> encryption format should be preferred for its simplicity and auditability.
-----------------	---------------	--

4. External Interface Requirements

4.1 User Interfaces

- Primary: REPL (existing, extended by this SRS)
- Secondary: Optional TUI (see REQ-UX-004 through REQ-UX-009)
- All terminal output shall respect the `NO_COLOR` environment variable standard

4.2 Software Interfaces

- **Model providers:** OpenAI-compatible chat completions API (streaming and non-streaming)
- **MCP servers:** stdio (local process) and HTTP (Streamable HTTP transport) as currently implemented
- **OS keyring (optional):** via platform-native APIs for encryption key storage
- **Git (optional):** for `enchanter skill add <git-url>` — invoked as a subprocess or via a Rust git library

4.3 Communication Interfaces

- Outbound HTTPS to model provider APIs
- Outbound HTTPS to MCP HTTP servers
- Local IPC to MCP stdio servers (stdin/stdout of spawned processes)

5. Non-Functional Requirements

REQ-ID	Category	Keyword	Requirement
REQ-NFR-001	Performance	SHALL	Prompt assembly and memory loading shall complete in under 200ms on typical local disk hardware.
REQ-NFR-002	Portability	SHALL	The binary shall be statically linked with zero runtime dependencies on Linux and macOS.
REQ-NFR-003	Portability	SHOULD	Windows (native, not WSL2) should be supported as a stretch goal with documented limitations.
REQ-NFR-004	Security	SHALL	Secrets (API keys, tokens, auth headers) shall never appear in terminal output, logs, or recordings by default.
REQ-NFR-005	Security	SHALL	All dependencies shall be MIT, Apache-2.0, or BSD-compatible licensed.
REQ-NFR-006	Reliability	SHALL	MCP stdio server crashes shall trigger auto-restart (up to 3 attempts) as currently implemented; this behavior shall not regress.
REQ-NFR-007	Reliability	SHALL	Feature failures (eval, TUI, encryption) shall degrade gracefully without crashing the core REPL.

SHALL

REQ- NFR-008	Backwards Compat.		Existing <code>~/.enchanter/</code> directory layouts shall continue to work without migration steps after any update.
REQ- NFR-009	Binary Size	SHOULD	TUI, eval, and encryption features should be gated behind Cargo feature flags to keep the default binary lean.
REQ- NFR-010	Testability	SHALL	Each module defined in Section 6 shall have unit tests covering its core logic. Integration tests shall cover the full prompt assembly pipeline.
REQ- NFR-011	Documentation	SHALL	All new CLI flags, REPL commands, config keys, and file formats introduced by this SRS shall be documented in the project README or linked docs before the feature is considered complete.
REQ- NFR-012	Code Quality	SHALL	No <code>unsafe</code> Rust code shall be introduced without a documented justification and a maintainer review comment in the PR.

6. Suggested Architecture

The following module structure is a recommendation to guide contributors. The goal is clean, testable boundaries that preserve the "focused harness" promise. Maintainers may adapt this structure as the codebase evolves.

6.1 Module Map

Module	Responsibility	Key Types / Traits
config	Resolves configuration from env vars, project overlay, global config, and defaults. Handles redaction.	ResolvedConfig , ConfigSource , Redactor
prompt	Assembles layered system prompt (SOUL / CONTEXT / VOLATILE). Provides diff and budget analysis.	PromptBuilder , PromptLayer , PromptDiff , BudgetReport
model_client	OpenAI-compatible HTTP client. Handles streaming and non-streaming responses. Provider switching.	ModelClient , ChatRequest , ChatResponse , StreamChunk
memory	Loads, parses, and writes memory files (\$ format). Manages channels, pin/redact/promote, summarization hook.	MemoryStore , MemoryEntry , MemoryChannel
skills	Discovers, indexes, and validates SKILL.md files. Manages skill package add/remove/doctor.	SkillIndex , Skill , SkillMetadata
tools	Unified tool registry combining local built-in tools and MCP tools. Dispatches tool calls.	ToolRegistry , Tool , ToolCall , ToolResult
mcp	MCP stdio and HTTP transports. Session tracking, retry policy, server lifecycle management.	McpServer , McpTransport , McpSession
policy	Capability definitions, consent store, decision engine. Enforces policies before tool dispatch.	PolicyEngine , Capability , ConsentDecision , ConsentStore
recorder	JSONL event writer. Redaction pipeline. Schema versioning.	Recorder , RecordEvent , RedactionPipeline
replay	JSONL event reader. Stubbed tool output injection. Diff output against original.	Replayer , ReplayMode , StubbedToolStore
eval	Built-in eval task harness. Scoring. Scorecard output (table + JSON). Comparison.	EvalRunner , EvalTask , Scorecard
ui::repl		Repl , Command , ReplOutput

	REPL loop, command parsing, output rendering.	
<code>ui::tui</code>	Optional TUI (feature-gated). Pane layout, keyboard shortcuts, live updates.	<code>TuiApp</code> , <code>Pane</code> , <code>KeyBinding</code>
<code>overlay</code>	Project overlay discovery, merge strategy, scope switching.	<code>OverlayResolver</code> , <code>Scope</code> , <code>MergeStrategy</code>

6.2 Data Flow



6.3 Cargo Feature Flags

Feature Flag	Enables	Default
<code>tui</code>	Optional TUI mode (<code>ui::tui</code> module, <code>ratatui</code> dependency)	Off
<code>eval</code>	Harness eval suite (<code>eval</code> module)	Off
	Optional memory encryption at rest (age crate)	Off

memory-encryption		
full	All of the above	Off

7. Suggested Libraries (Rust Crates)

All suggested crates are MIT or Apache-2.0 licensed. Final crate selection is at maintainer discretion; this list is a starting point for contributors.

Purpose	Suggested Crate(s)	Notes
CLI argument parsing	<code>clap</code> (derive feature)	Already likely in use; supports subcommands cleanly.
Async runtime	<code>tokio</code>	Full features for HTTP + stdio MCP concurrency.
HTTP client	<code>reqwest</code>	TLS, streaming, async. Used for model API + MCP HTTP.
Serialization	<code>serde</code> , <code>serde_json</code> , <code>serde_yaml</code>	JSONL recording, config YAML, skill metadata.
Error handling	<code>anyhow</code> (app), <code>thiserror</code> (library)	Use <code>thiserror</code> for module-level errors; <code>anyhow</code> at the top level.
Logging / tracing	<code>tracing</code> , <code>tracing-subscriber</code>	Structured logging; useful for debugging MCP and recorder.
Config directory resolution	<code>directories</code>	Cross-platform <code>~/.enchanter/</code> resolution.
Secret handling	<code>secrecy</code>	Wraps secrets to prevent accidental <code>Debug</code> / <code>Display</code> leaks.
Text diff (prompt diff)	<code>similar</code>	Inline and unified diff; good terminal output support.
Token estimation	Custom heuristic (chars / 4)	Keep lightweight; avoid tokenizer bloat. Label output as approximate.
Terminal UI (optional)	<code>ratatui</code> + <code>crossterm</code>	Feature-gated. Ratatui is the de facto standard for Rust TUIs.
REPL line editing	<code>rustyline</code>	History, completion, Emacs/Vi keybindings.
Git operations (skill add)	<code>git2</code> or <code>subprocess</code> <code>git</code>	<code>git2</code> keeps binary self-contained; <code>subprocess</code> is simpler but requires git installed.
Encryption at rest (optional)	<code>age</code>	Feature-gated. Simple, auditable, widely trusted format.
OS keyring (optional)	<code>keyring</code>	Cross-platform keyring access for encryption key storage.
Checksum / pinning	<code>sha2</code>	For skill package integrity verification.
Table output	<code>comfy-table</code>	Clean terminal tables for eval scorecards, skill list, budget view.

YAML front-matter parsing `gray_matter` or custom For SKILL.md metadata front-matter.

8. File & Format Specifications

8.1 Directory Structure

```
~/enchanter/                # Global home (existing)
  SOUL.md
  config.yaml
  memories/
  skills/
  consent.yaml              # NEW: global consent decisions
  audit.log                 # NEW: append-only audit log

.enchanter/                 # NEW: project overlay (in project root)
  SOUL.md                   # optional addendum/override
  config.yaml               # optional overlay
  memories/
  skills/
  consent.yaml              # project consent decisions
```

8.2 Run Recording Format (JSONL)

Each line is a JSON object with the following envelope:

```
{
  "schema_version": "1",
  "seq": 1,
  "ts": "2026-05-24T12:00:00Z",
  "type": "user_message" | "assistant_response" | "tool_call" |
         "tool_result" | "memory_write" | "config_snapshot" |
         "session_summary" | "consent_decision",
  "payload": { ... }
}
```

8.3 Consent Store Format (YAML)

```
decisions:
- tool: filesystem
  capability: write
  paths: ["/home/user/projects"]
  decision: allow_project
  ts: "2026-05-24T12:00:00Z"
```

8.4 SKILL.md Metadata Front-Matter (Proposed)

```
---
name: web-search
version: 1.0.0
required_tools: [fetch]
```

```
description: Search the web and summarize results.
examples:
  - "Search for recent Rust async news"
---
# Web Search Skill
... skill body ...
```

8.5 Eval Scorecard Format (JSON)

```
{
  "schema_version": "1",
  "ts": "2026-05-24T12:00:00Z",
  "model": "qwen3",
  "provider": "ollama",
  "config_hash": "abc123",
  "results": [
    { "task": "memory_recall_short", "score": 1.0, "pass": true },
    { "task": "skill_selection", "score": 0.8, "pass": true }
  ],
  "summary": { "total": 5, "passed": 4, "score": 0.88 }
}
```

9. Milestones & Traceability Matrix

9.1 Milestones

Milestone	Target Features	Notes
MVP	Prompt diff + budget (REQ-INS-001–010), Record + replay (REQ-REC-001–006, REQ-REP-001–006), Project overlay (REQ-OVR-001–007), Basic consent prompts + path allowlist (REQ-SEC-001–009)	Core differentiators. Establishes Enchanter's inspectability and reproducibility story.
v1.0	Skill metadata + management (REQ-SKL-001–009), Eval suite + scorecards (REQ-EVL-001–008), Expanded policy engine + audit log (REQ-SEC-010–011), Memory channels + governance commands (REQ-MEM-001–008)	Ecosystem and quality features. Enables community skill sharing and harness tuning.
v1.5	Optional TUI (REQ-UX-004–009), Optional memory encryption (REQ-MEM-009–012), Windows stretch goal (REQ-NFR-003)	Polish and power-user features. Feature-gated to keep default binary lean.

9.2 Requirements Traceability Matrix

REQ-ID	Feature Area	Milestone	Keyword
REQ-INS-001–005	Prompt Diff	MVP	SHALL / SHOULD
REQ-INS-006–010	Prompt Budget	MVP	SHALL / SHOULD / MAY
REQ-REC-001–006	Recording	MVP	SHALL / SHOULD
REQ-REP-001–006	Replay	MVP	SHALL / SHOULD
REQ-OVR-001–007	Project Overlays	MVP	SHALL / SHOULD / MAY
REQ-SEC-001–009	MCP Policy + Consent	MVP	SHALL
REQ-SEC-010–011	Audit Log	v1.0	SHOULD / MAY
REQ-SKL-001–009	Skill Packages	v1.0	SHALL / SHOULD / MAY
REQ-EVL-001–008	Harness Evals	v1.0	SHALL / SHOULD / MAY
REQ-UX-001–003	REPL Enhancements	MVP	SHALL / SHOULD
REQ-UX-004–009	Optional TUI	v1.5	SHALL / SHOULD
REQ-MEM-001–008	Memory Channels + Governance	v1.0	SHALL / SHOULD
REQ-MEM-009–012	Memory Encryption	v1.5	MAY / SHALL / SHOULD
REQ-NFR-001–012	Non-Functional	All	SHALL / SHOULD

10. Open Questions

The following questions require community discussion and maintainer decisions before implementation begins. Open a GitHub Issue referencing the question number to discuss.

OQ-001 — Determinism target: Do we aim for "prompt determinism" (same harness inputs → same assembled prompt) or "response determinism" (same prompt → same model response)? Only the former is realistically achievable across providers. Recommendation: document that Enchanter guarantees prompt determinism only; response determinism is provider-dependent.

OQ-002 — Overlay merge strategy: When project and global overlays define the same skill name or memory ID, the current proposal is "project takes precedence." Should there be a merge mode that combines both? What is the conflict resolution for MCP server names?

OQ-003 — Consent UX in non-interactive mode: REQ-SEC-009 defaults to Deny for unapproved tools in `-p` mode. Should there be a `--allow-all-tools` escape hatch for scripting? If so, what are the security implications and documentation requirements?

OQ-004 — Skill packaging trust model: Should skill packages require cryptographic signatures, SHA-256 checksums, or pinned git commits? What is the minimum viable trust model for the MVP vs. v1.0?

OQ-005 — Windows support: Is native Windows (non-WSL2) a v1.5 goal or out of scope? Key blockers: static linking, MCP stdio process spawning, keyring integration, and TUI terminal compatibility.

OQ-006 — Memory format evolution: The current `$`-delimited format is inherited from Hermes Agent. Should Enchanter define its own extended format (with channel metadata) while maintaining read compatibility with the original format? Or should channels be stored as a sidecar index?

OQ-007 — Eval task contribution: REQ-EVL-007 proposes community-contributed eval tasks via a drop-in mechanism. What is the security model for running community eval tasks? Should they be sandboxed?

OQ-008 — Token estimation accuracy: The proposed heuristic (`chars / 4`) is a rough approximation. Should Enchanter optionally support provider-specific tokenizers (e.g., tiktoken via FFI or a pure-Rust port) as an opt-in feature? What is the binary size impact?

11. Contributing

Enchanter is an open-source project under the MIT license. All contributions are welcome.

11.1 How to Contribute

- **Issues:** Open a GitHub Issue for bugs, feature requests, or discussion of Open Questions (Section 10). Reference REQ-IDs where applicable.
- **Pull Requests:** Reference the REQ-ID(s) your PR addresses in the PR description. Include tests for new functionality (see REQ-NFR-010).

- **Skills:** Share SKILL.md files via Issues or PRs. Once REQ-SKL-005 is implemented, git-URL-based skill sharing will be the preferred mechanism.
- **Eval Tasks:** Propose new harness eval tasks via Issues once the eval framework (REQ-EVL-001) is in place.
- **Documentation:** All new features require documentation before a PR is considered complete (REQ-NFR-011).

11.2 Code Standards

- Run `cargo fmt` and `cargo clippy` before submitting a PR.
- No `unsafe` code without documented justification and maintainer review (REQ-NFR-012).
- All new dependencies must be MIT, Apache-2.0, or BSD-compatible (REQ-NFR-005).
- Feature-gated code must compile cleanly with and without the feature flag.

11.3 Positioning Statement

"The most inspectable, reproducible, zero-dependency agent harness for the terminal — bring your own model, tools (MCP), and skills (markdown)."